CS311 - Design & Analysis of Algorithms

Graph and Search Algorithms

Graphs

- A ***graph*** is a pair $G = (V, E)$, where
  - V *is a set*
  - E *is a set of pairs* (v, w) *with* v, w $\in$ V
- We call the elements of V the ***vertices*** of G;
- the elements of E are the ***edges*** of G.

Example:

$V = \{a, b, c, d, e, f, g\}$
$E = \{(a, c), (b, d), (b, e), (b, g), (d, e), (d, g), (e, f)\}$

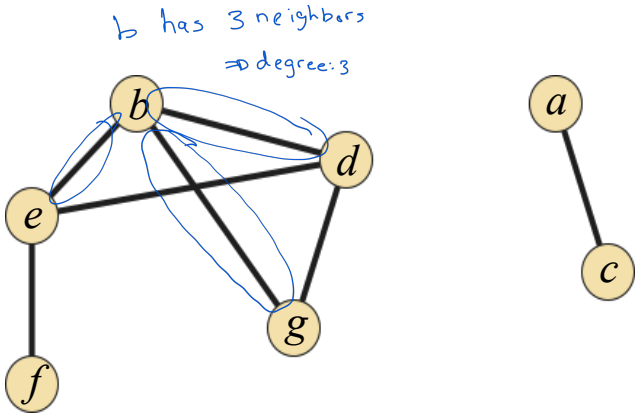
Vertices a and c are ***adjacent***;
vertices f and g are not.

Vertex b and edge (b, g) are ***incident***; vertex e and edge (d, g) are not.

A vertex and an edge are called incident, if the vertex is one of the two vertices the edge connects.

The ***neighbourhood*** of vertex b is the set $N(b) = \{d, e, g\}$.

The ***degree*** $\deg(b)$ of vertex b is 3.



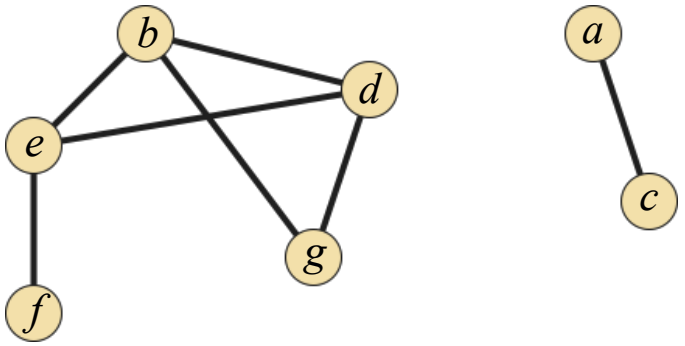
Let G be an undirected graph with edge set E.
Let $e \in E$ where $e = \{u,v\}$. Then we say:

- u, v are ***adjacent*** / ***neighbors*** / ***connected***.
- **Edge** e **is incident with** vertices u and v.
- **Edge** e **connects** u and v.
- **Vertices** u and v are ***endpoints*** of edge e.
- The *degree* of v, **deg(v)**, is its number of incident edges. (**Except that** any self-loops are counted twice.)
- A **vertex** with **degree 0** is *isolated*.
- A **vertex** of **degree 1** is *pendant*.

Undirected vs. Directed Graphs

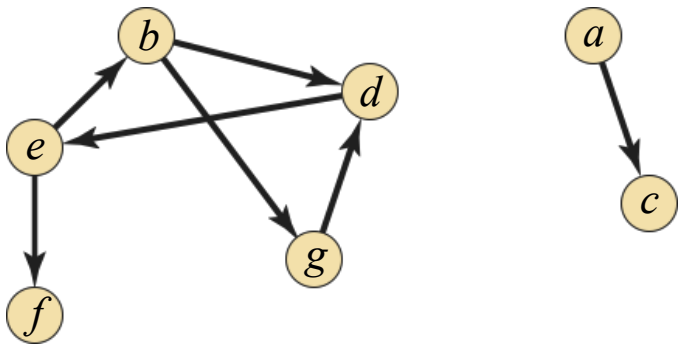
In an undirected graphs, edges are ***unordered pairs*** (v, w).

That is, edges (v, w) and (w, v) are the same edge; there is no sense of direction.



In a directed graph, edges are ***ordered pairs*** (v, w).

That is, edges (v, w) and (w, v) are two different edges.
Edge (v, w) is directed from v to w.
Vertex v is the ***source*** of edge (v, w); vertex w is its ***target***.

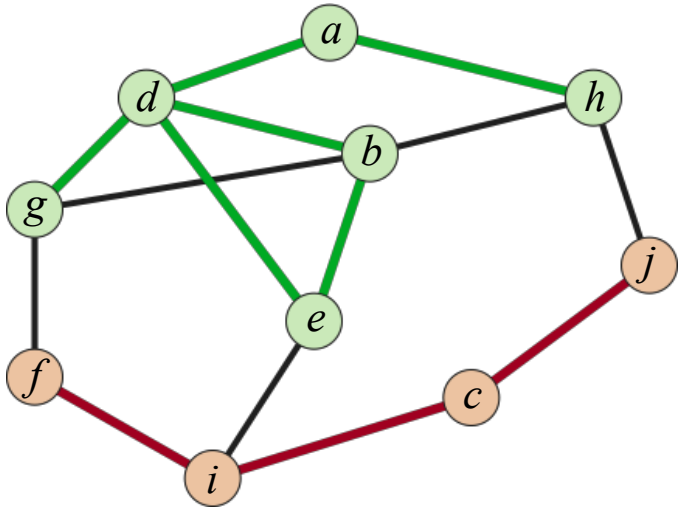


Paths and Cycles

A ***path*** is a sequence of vertices $P = (v_0, v_1, \ldots, v_k)$ such that, for $1 \leq i \leq k$, edge $(v_{i-1}, v_i) \in E$.

Path P is ***simple*** if no vertex appears more than once in P.

$P_1 = (g, d, e, b, \underline{d}, a, h)$ is not simple.



$P_2 = (f, i, c, j)$ is simple.

Paths and Cycles

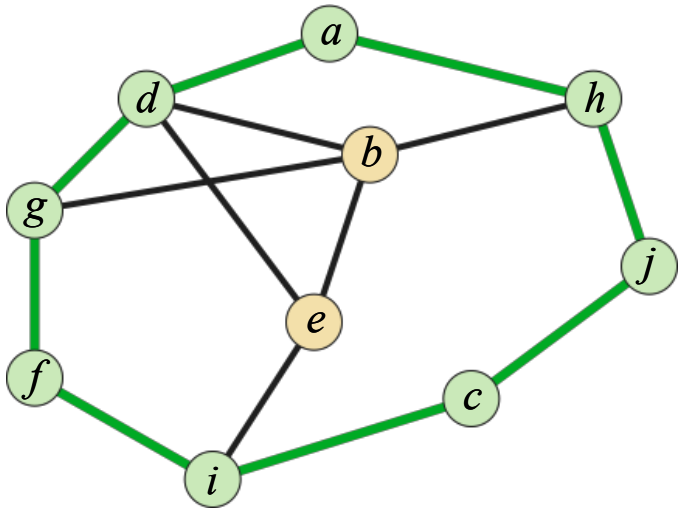
A ***path*** is a sequence of vertices $P = (v_0, v_1, \ldots, v_k)$ such that, for $1 \le i \le k$, edge $(v_{i-1}, v_i) \in E$.

Path P is ***simple*** if no vertex appears more than once in P.

A ***cycle*** is a sequence of vertices $C = (v_0, v_1, \ldots, v_{k-1})$ such that, for $0 \le i < k$, edge $(v_i, v_{(i+1) \bmod k}) \in E$.

Cycle C is ***simple*** if path $(v_0, v_1, \ldots, v_{k-1})$ is simple.
A simple cycle is a cycle with no repeated vertices (except for the beginning and ending vertex).

$C_1 = (a, h, j, c, i, f, g, d)$ is simple.



Paths and Cycles

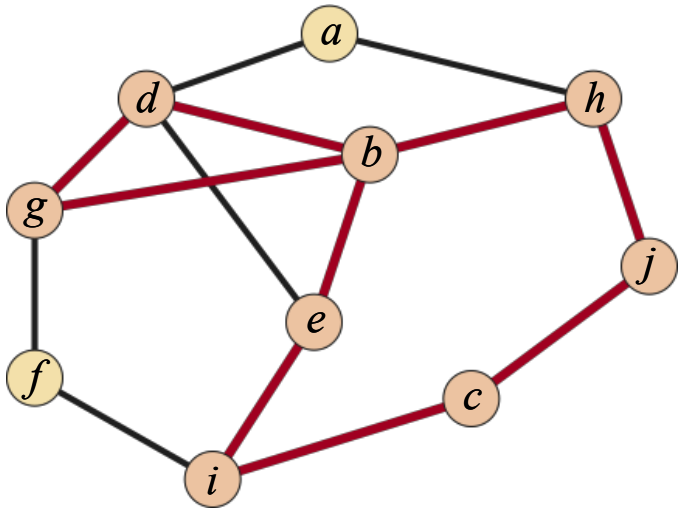
A ***path*** is a sequence of vertices $P = (v_0, v_1, \ldots, v_k)$ such that, for $1 \le i \le k$, edge $(v_{i-1}, v_i) \in E$.

Path P is ***simple*** if no vertex appears more than once in P.

A ***cycle*** is a sequence of vertices $C = (v_0, v_1, \ldots, v_{k-1}, v_0)$ such that, for $0 \le i < k$, edge $(v_i, v_{(i+1) \bmod k}) \in E$.

Cycle C is ***simple*** if path $(v_0, v_1, \ldots, v_{k-1})$ is simple.
A simple cycle is a cycle with no repeated vertices (except for the beginning and ending vertex).

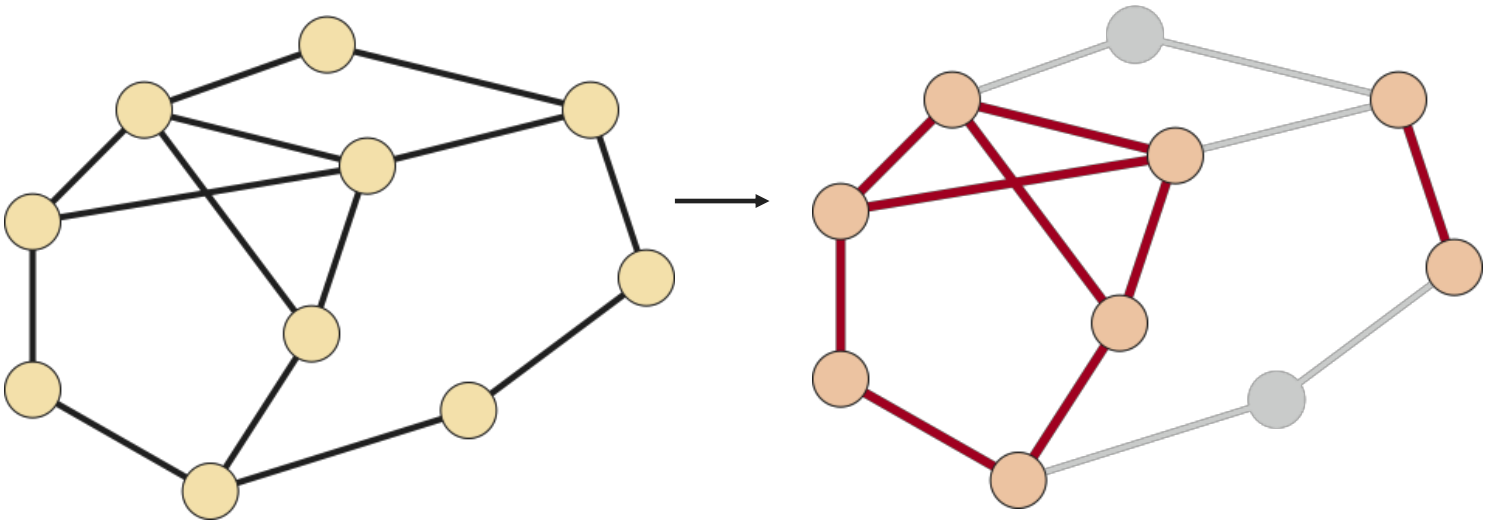
$C_1 = (a, h, j, c, i, f, g, d)$ is simple.



$C_2 = (g, d, \underline{b}, h, j, c, i, e, \underline{b})$ is not simple.

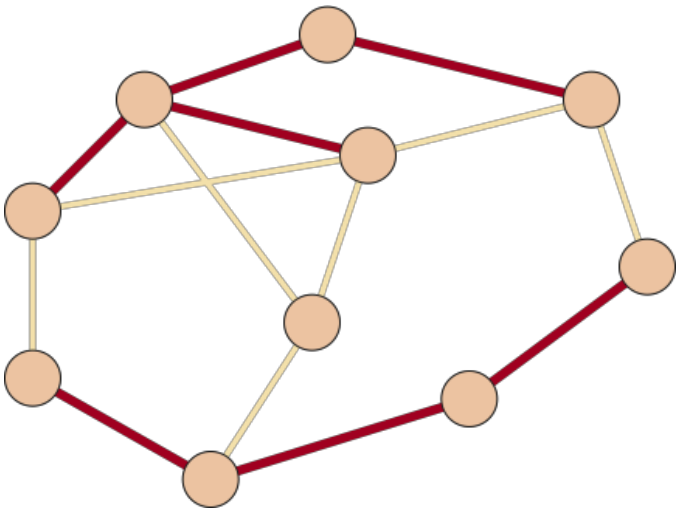
Subgraphs

- A graph $H = (W, F)$ is a ***subgraph*** of a graph $G = (V, E)$ if $W \subseteq V$ and $F \subseteq E$.



Spanning Graphs

A ***spanning graph*** of G is a subgraph of G that contains **all** vertices of G.



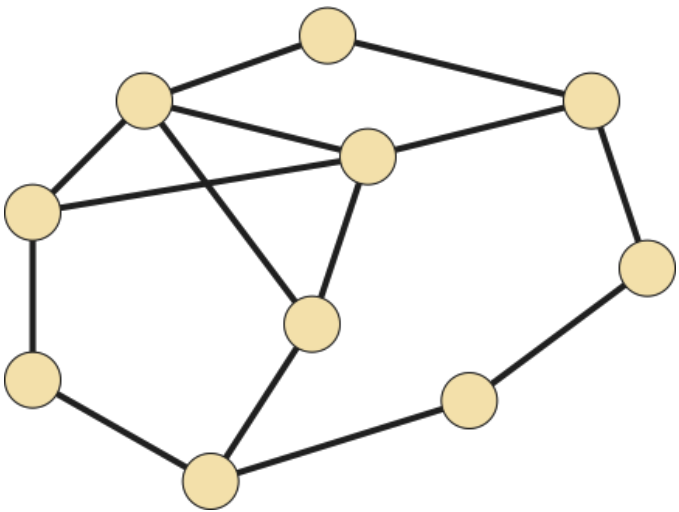
Connectivity

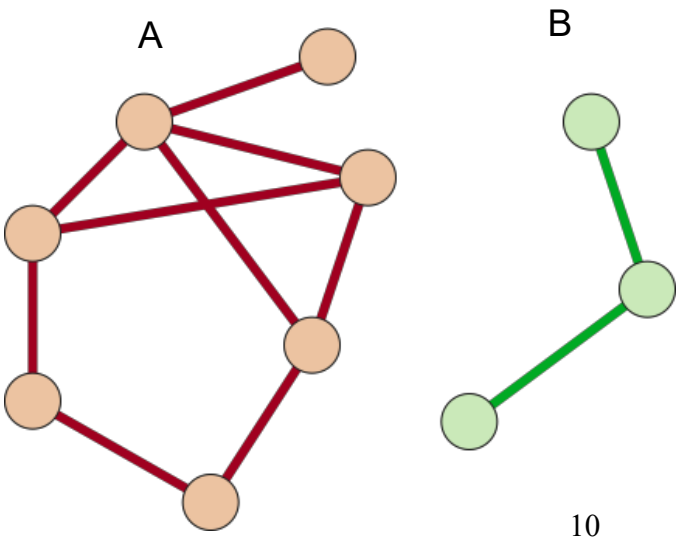
A graph *G* is ***connected*** if there is a path between any two vertices in *G*.

The ***connected components*** of a graph are its maximal connected subgraphs.

Given a graph G=(V, E), a subgraph S=(V', E') is a maximally connected component if

- S is connected and
- for all vertices u such that u∈ V and u∉ V' there is no vertex v∈ V' for which (u, v)∈ E



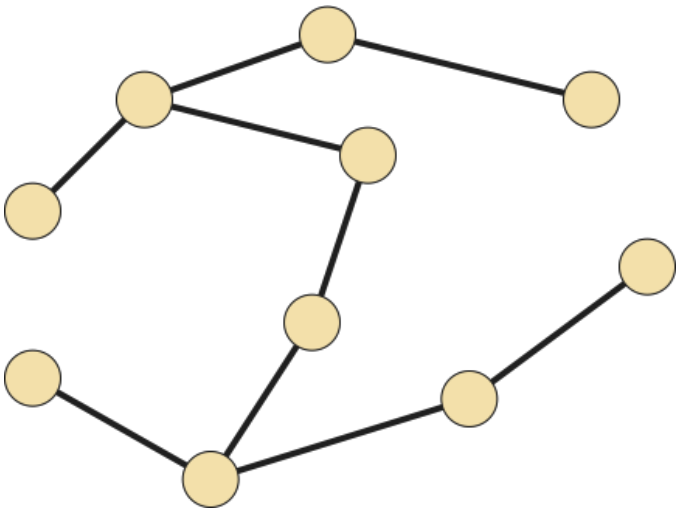


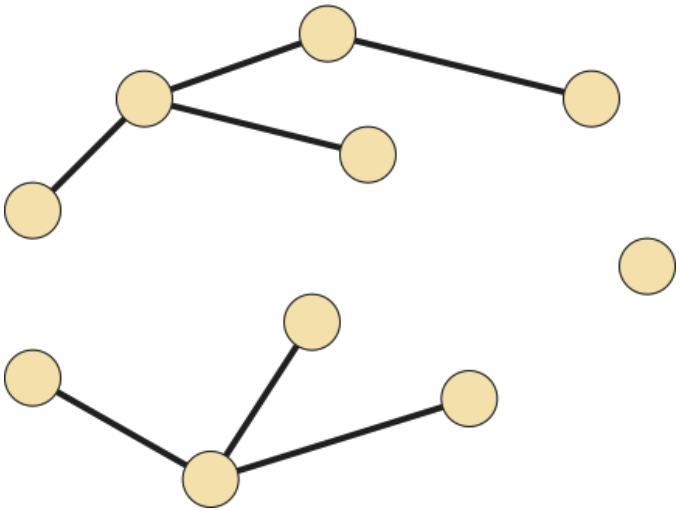
Trees and Forests

A ***tree*** is a graph that is connected and contains no cycles.

A ***forest*** is a graph that contains no cycles but not necessarily connected.

The connected components of a forest are trees.

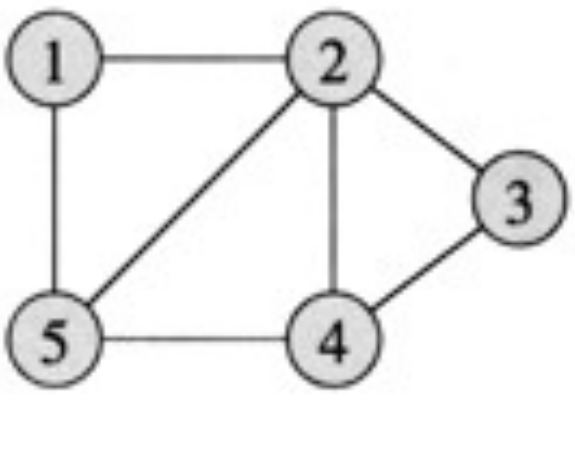




Graph Representations

Adjacency matrix representation

- Better with dense graph, Space complexity $O(V^2)$
- Provide the Adjacency matrix to represent the following graph:



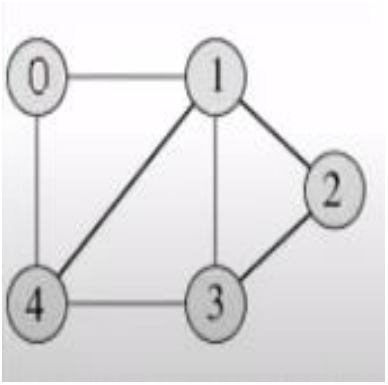
	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

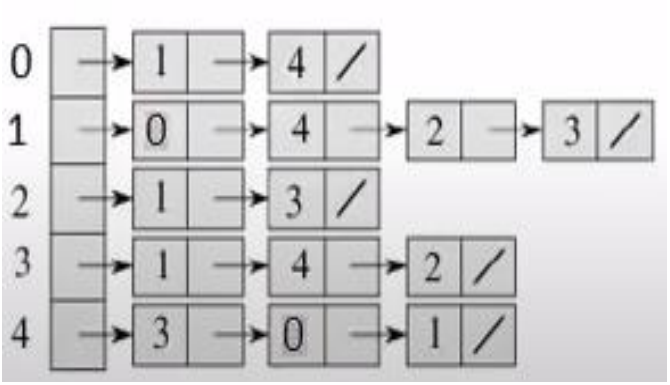
Graph Representations

Adjacency List representation

Sparse graph

Space complexity = O(V+E)





Graph Exploration *

- **Goal:** Visit all the vertices in the graph; identify the connected components; compute a spanning tree of *G*.

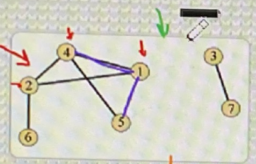
There could be more than one spanning tree

```
ExploreGraph(G)
1     for every vertex v of G
2           do label v as unexplored
3     for every edge e of G
4           do label e as unexplored
5     for every vertex v of G
6           do if v is unexplored
7           then mark v as visited
8           S ← Adj(v)
9           while S is not empty
10                do remove an edge (u, w) from S
11                if (u, w) is unexplored
12                   then if w is unexplored   if w is explored, dont add it in the spanning tree as it will make a loop
13                   then mark edge (u, w) as tree edge
14                      mark vertex w as visited
15                      S ← S ∪ Adj(w)
16                   else mark edge (u, w) as a non-tree edge
```

In the final, they could ask to write this code

Untitled Free

```
ExploreGraph(G)
1  for every vertex v of G
2      do label v as unexplored
3  for every edge e of G
4      do label e as unexplored
5  for every vertex v of G
6      do if v is unexplored
7         then mark v as visited
8              S ← Adj(v)
9              while S is not empty
10                 do remove an edge (u, w) from S
11                 if (u, w) is unexplored
12                    then if w is unexplored
13                            then mark edge (u, w) as tree edge
14                                 mark vertex w as visited
15                            S ← S ∪ Adj(w)
16                    else mark edge (u, w) as a non-tree edge
```



V unexplored: {~~1~~, ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~, ~~7~~}

E unexplored: {~~(1,4)~~, ~~(1,2)~~, ~~(1,5)~~, ~~(2,4)~~ ~~(3,1)~~ ~~(2,6)~~, ~~(3,7)~~, ~~(4,1)~~ ~~(4,2)~~ ~~(4,5)~~ ~~(5,1)~~ ~~(5,4)~~ ~~(6,2)~~ ~~(7,3)~~}

Visited = {1, 5, 4, 2, 6, 3, 7}

find connected component & create spanning tree.

1 visited

S ← {~~(1,4)~~, ~~(1,2)~~, ~~(1,5)~~}

remove (1,5)

5 unexplored

S ← ~~(1,4)~~ ~~(1,2)~~ (5,1) (5,4)

remove (1,4)

4 unexpled

S ← ~~(1,4)~~ (5,1) (5,4) (4,1) (4,5) (4,2)

remove (1,2)

2 visited

S ← ~~(5,1)~~ (5,4) ~~(4,1)~~ ~~(4,5)~~ ~~(4,2)~~ ~~(2,4)~~ ~~(2,1)~~ ~~(2,6)~~

remove (5,1)

1 explored

remove (5,4)

4 explored

remove (4,1)

1 exp.

remove (4,5)

5 exp.

remove (4,2)

2 exp.

remove (2,4)

(2,1)

remove (2,6)

6 unexp.

S ← (6,2)

remove (6,2)

2 exp.

3 unvisited

S ← ~~(3,4)~~

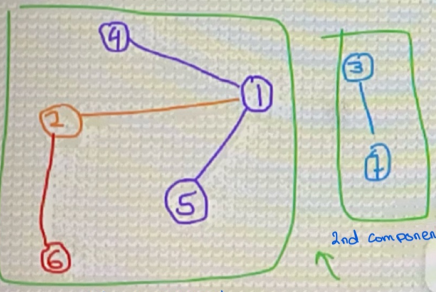
remove (3,7)

7 unexp.

S ← ~~(7,3)~~

remove (7,3)

3 visited



25°C غائم غالبًا

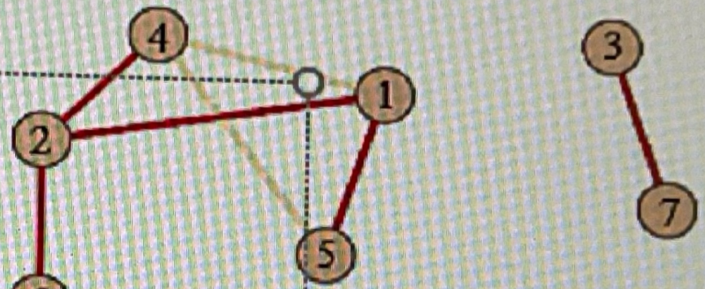
Search

ENG 9:34 AM 3/30/2026

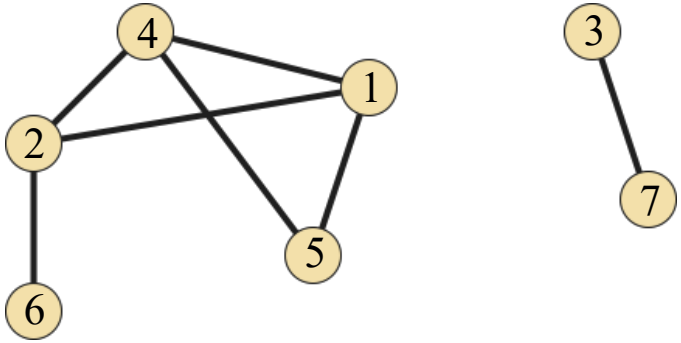
An Example: G=(V, E) n=|V| m=|E|

```
ExploreGraph(G)
1       for every vertex v of G
2           do label v as unexplored
3       for every edge e of G
4           do label e as unexplored
5  counter = 0;
for every vertex v of G
Counter ++;
6           do if v is unexplored
7           then mark v as visited
8           S ← Adj(v)
9           while S is not empty
10              do remove an edge (v, u) from S
11              if (v, u) is unexplored
12                  then if u is unexplored
13                  then mark edge (v, u) as tree edge
14                          mark vertex u as visited
15                          S ← S ∪ Adj(u)
16                  else mark edge (v, u) as a non-tree edge
```

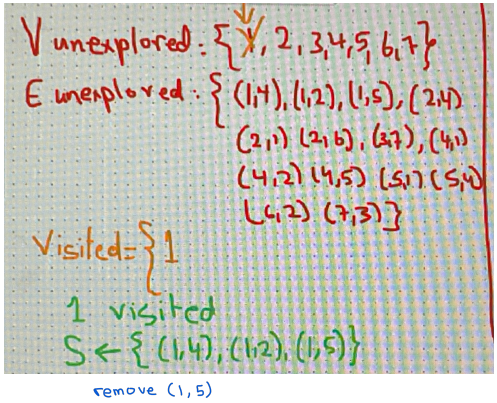
We could add a counter to see how many times we enter the for loop which would find the number of components in the graph



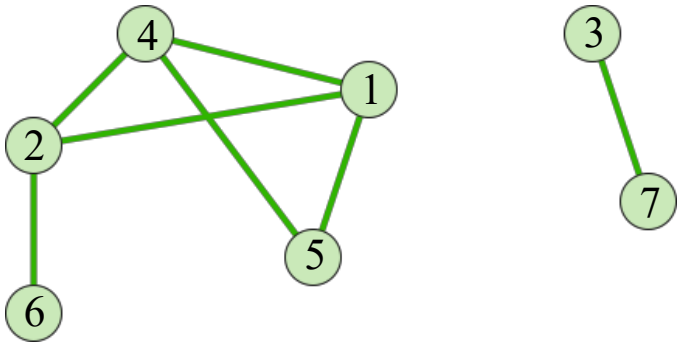
An Example




```
ExploreGraph(G)
1     for every vertex v of G
2         do label v as unexplored
3     for every edge e of G
4         do label e as unexplored
5     for every vertex v of G
6         do if v is unexplored
7         then mark v as visited
8         S ← Adj(v)
9         while S is not empty
10            do remove an edge (v, u) from S
11            if (v, u) is unexplored
12                then if u is unexplored
13                then mark edge (v, u) as tree edge
14                     mark vertex u as visited
15                     S ← S ∪ Adj(u)
16                else mark edge (v, u) as a non-tree edge
```



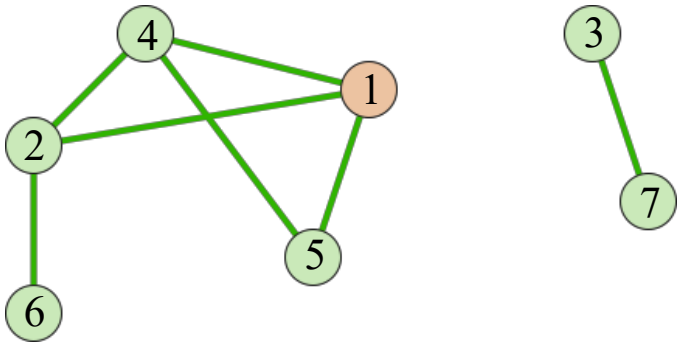
An Example




```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```

An Example

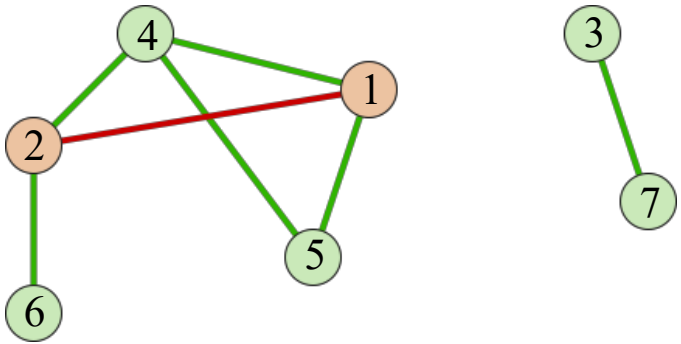
```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```



S: (1, 4) (1, 2) (1, 5)

An Example

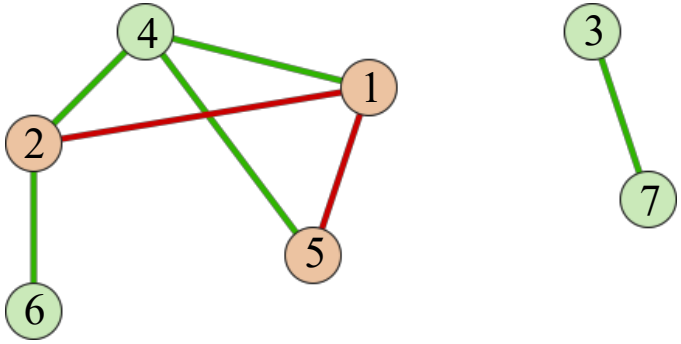
```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```



S: (1, 4) (1, 5) (2, 1) (2, 4) (2, 6)

An Example

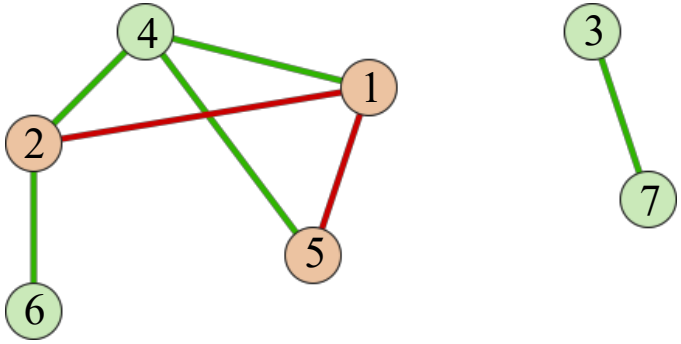
```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```



S: (1, 4) (2, 1) (2, 4)
(2, 6) (5, 1) (5, 4)

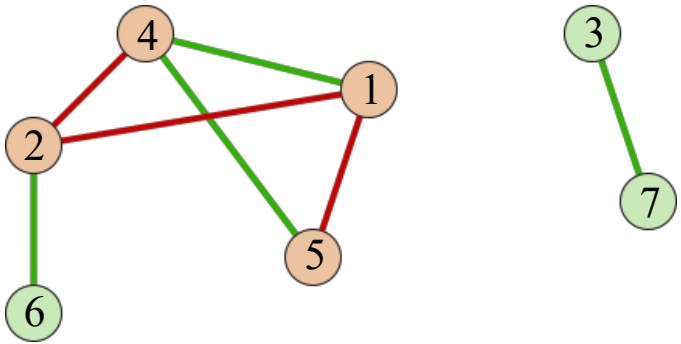
An Example

```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```



S: (1, 4) (2, 1) (2, 4) (2, 6) (5, 4)

An Example

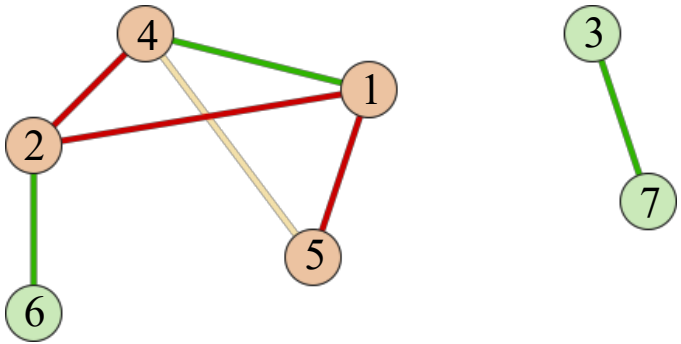



```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```

S: (1, 4) (2, 1) (2, 6) (5, 4) (4, 2) (4, 1) (4, 5)

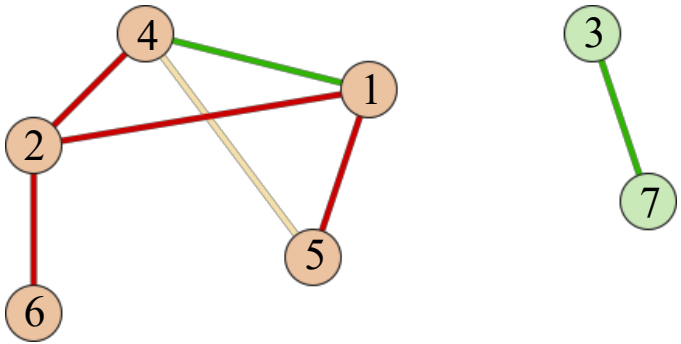
An Example

```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```



S: (1, 4) (2, 1) (2, 6)
(5, 4) (4, 2) (4, 1)

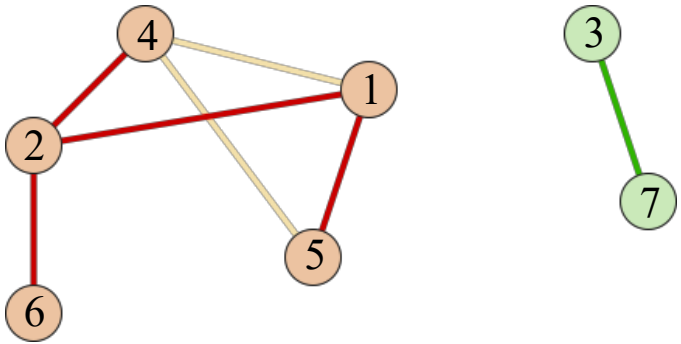
An Example




```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```

S: (1, 4) (2, 1) (5, 4)
(4, 2) (4, 1) (6, 2)

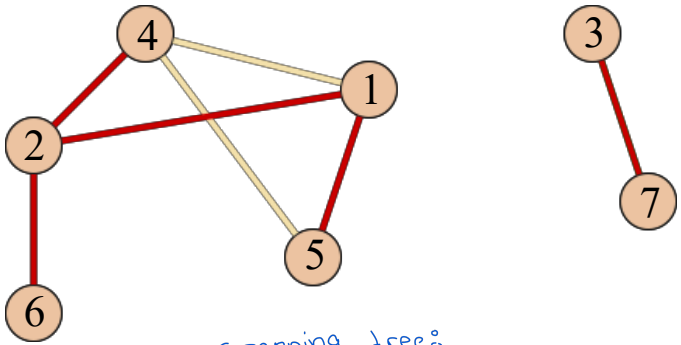
An Example



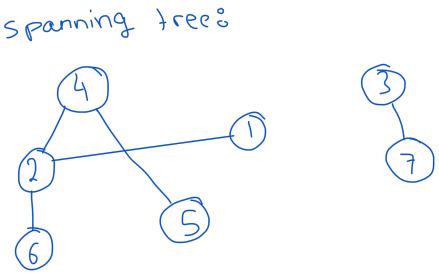

```
ExploreGraph(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do remove an edge (v, u) from S
11                      if (v, u) is unexplored
12                              then if u is unexplored
13                              then mark edge (v, u) as tree edge
14                                      mark vertex u as visited
15                                      S ← S ∪ Adj(u)
16                              else mark edge (v, u) as a non-tree edge
```

S: (1, 4) (2, 1) (5, 4) (4, 2) (6, 2)

An Example: G=(V, E) n=|V| m=|E|




```
ExploreGraph(G)
1     for every vertex v of G
2           do label v as unexplored
3     for every edge e of G
4           do label e as unexplored
5     for every vertex v of G
6           do if v is unexplored
7           then mark v as visited
8           S ← Adj(v)
9           while S is not empty
10                do remove an edge (v, u) from S
11                if (v, u) is unexplored
12                      then if u is unexplored
13                      then mark edge (v, u) as tree edge
14                            mark vertex u as visited
15                            S ← S ∪ Adj(u)
16                      else mark edge (v, u) as a non-tree edge
```



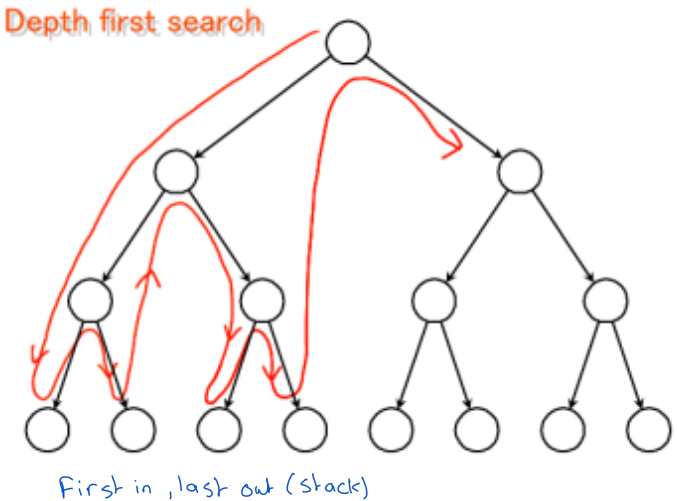
Properties of ExploreGraph

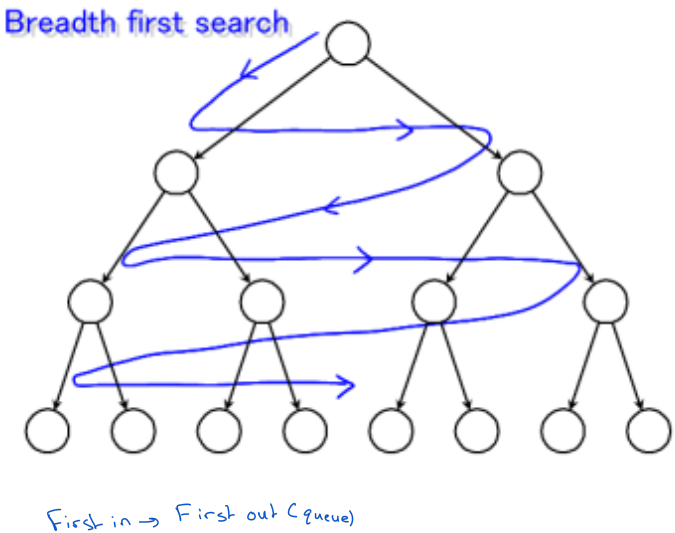
- **Theorem:** *Procedure ExploreGraph visits every vertex of G.*

- **Theorem:** *Every iteration of the for-loop explores a connected component of G.*

- **Theorem:** *The graph defined by the tree edges is a spanning forest of G.*

Graph Traversals





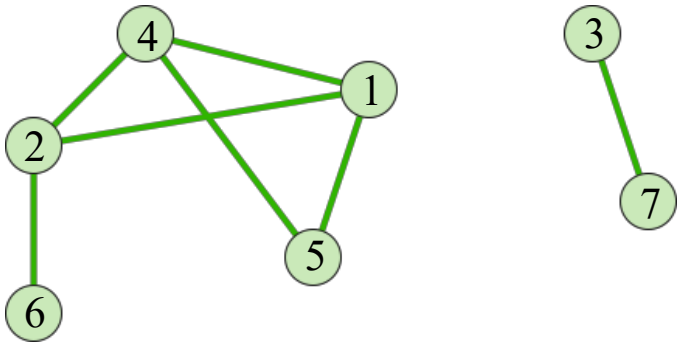
- Both take time: O(V+E)

Breadth-First Search

```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G                    Running time:  O(n + m)
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (v, w) ← Dequeue(S)
11                      if (v, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (v, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (v, w) as a non-tree edge
```

An Example

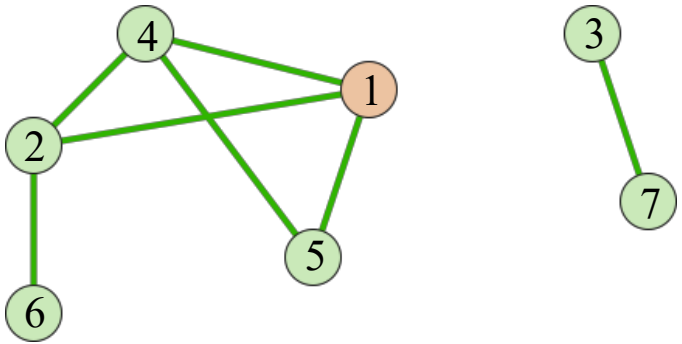
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (v, w) ← Dequeue(S)
11                      if (v, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (v, w) as a non-tree edge
```



An Example

(1, 4) (1, 5) (1, 2)

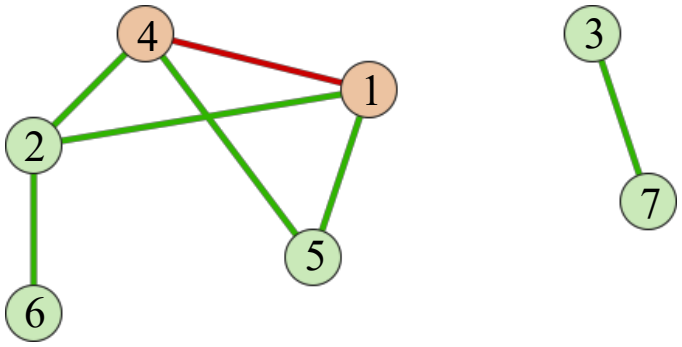
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(1, 5) (1, 2) (4, 1) (4, 5) (4, 2)

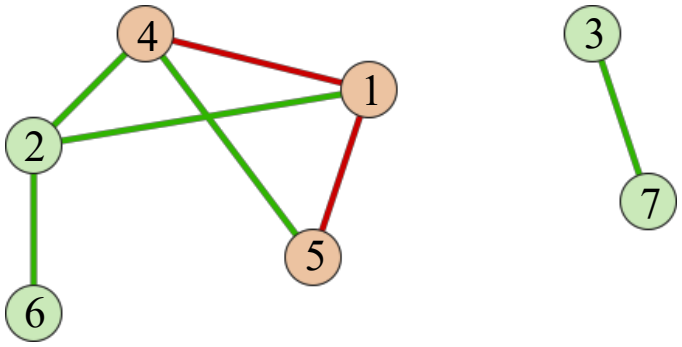
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(1, 2) (4, 1) (4, 5) (4, 2) (5, 1) (5, 4)

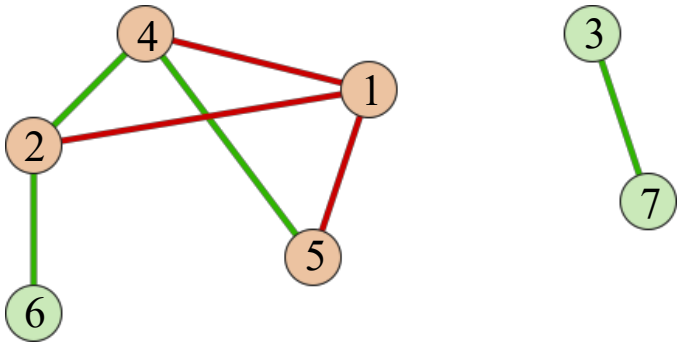
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(4, 1)	(4, 5)	(4, 2)	(5, 1)	(5, 4)	(2, 1)	(2, 4)	(2, 6)

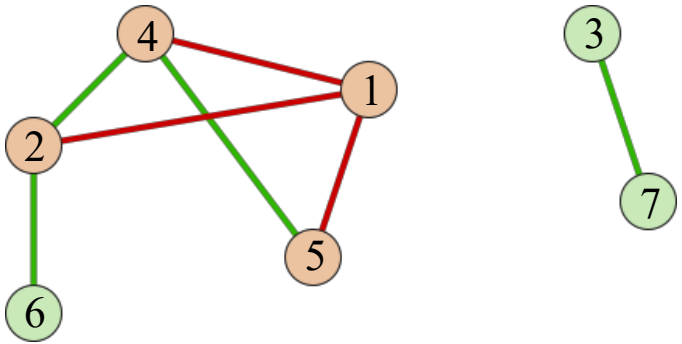
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(4, 5) (4, 2) (5, 1) (5, 4) (2, 1) (2, 4) (2, 6)

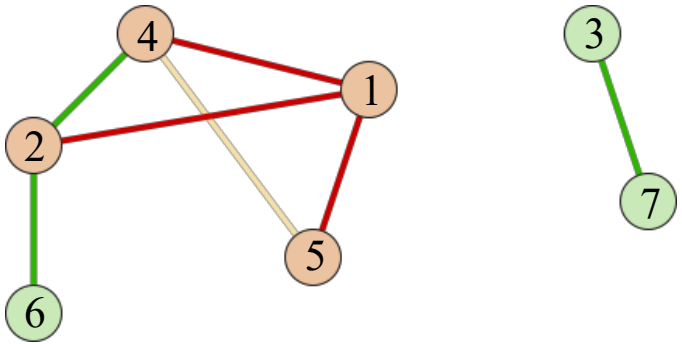
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(4, 2) (5, 1) (5, 4) (2, 1) (2, 4) (2, 6)

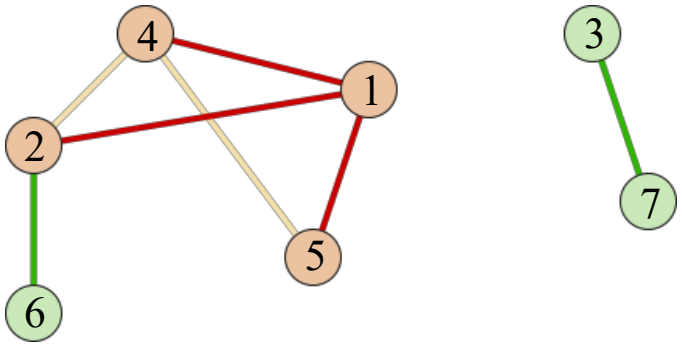
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(5, 1) (5, 4) (2, 1) (2, 4) (2, 6)

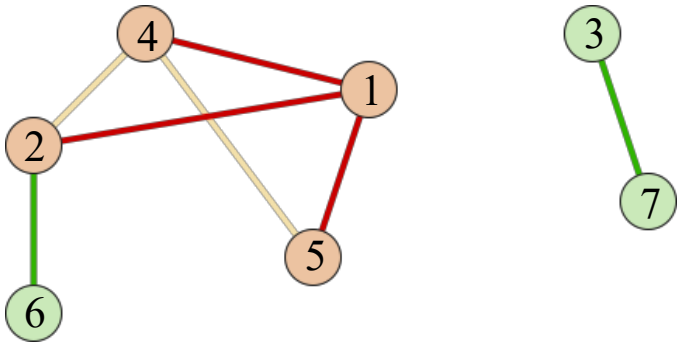
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

(2, 6)

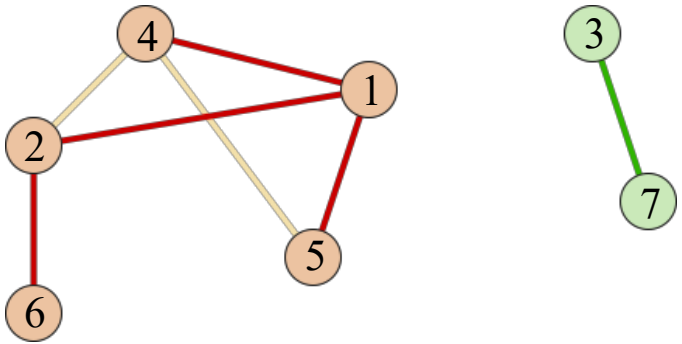
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

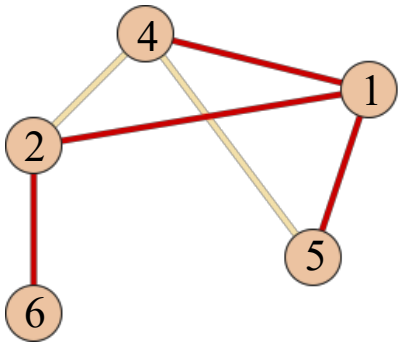
(6, 2)

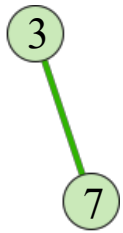
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



An Example

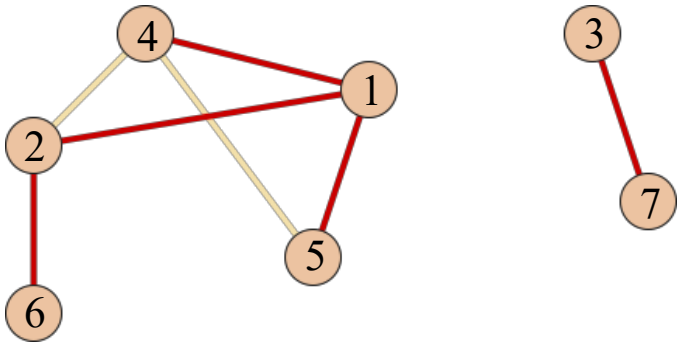
```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```





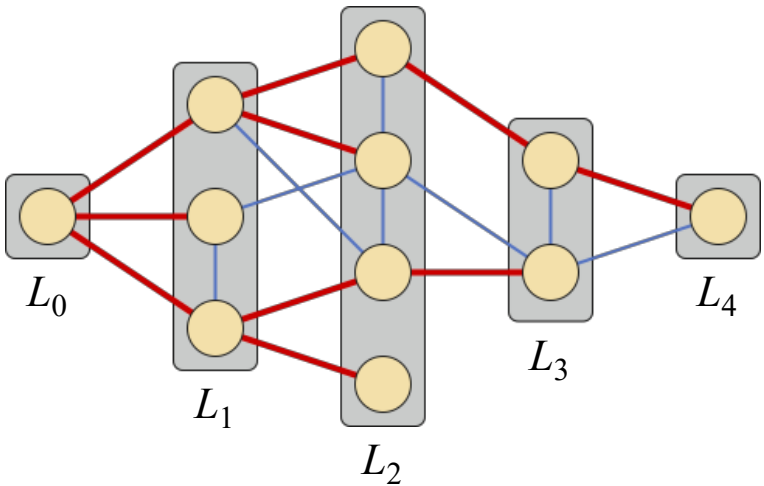
An Example

```
BFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
                do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Dequeue(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Enqueue(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



Properties of Breadth-First Search

Theorem: *Breadth-first search visits the vertices of G by increasing distance from the root.*

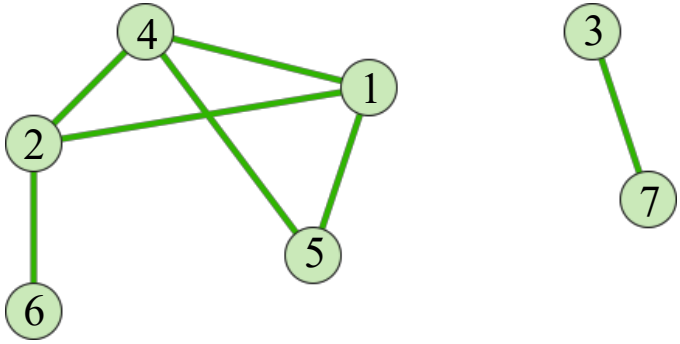


Theorem: *For every edge* (v, w) *in G such that* $v \in L_i$ *and* $w \in L_j$, $|i - j| \leq 1$.

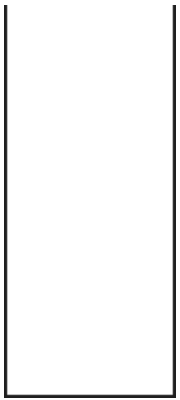
Depth-First Search

```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G                          Running time:  O(n + m)
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

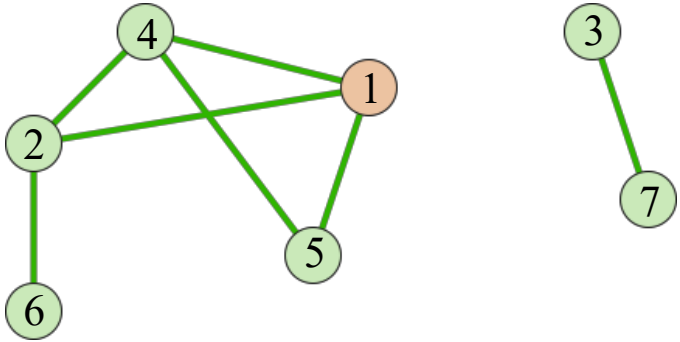
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



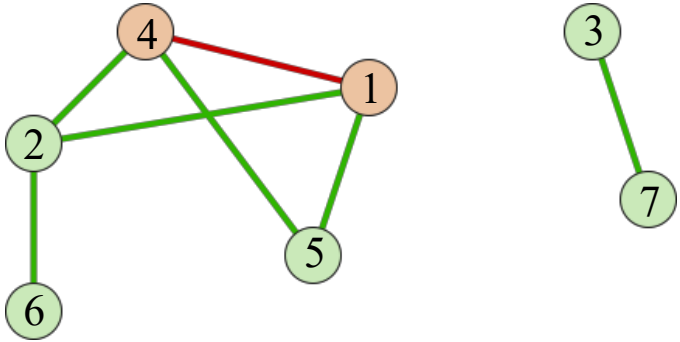
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(1, 4)
(1, 2)
(1, 5)

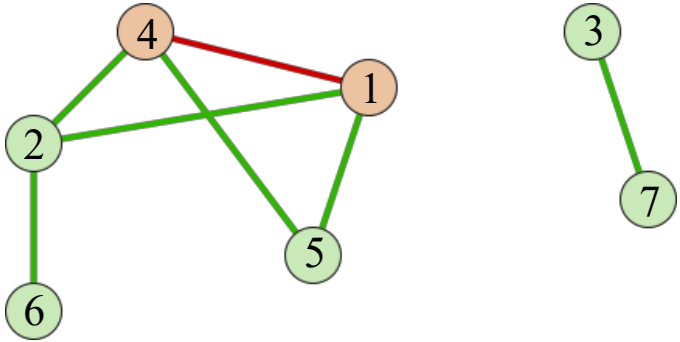
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(4, 1)
(4, 5)
(4, 2)
(1, 2)
(1, 5)

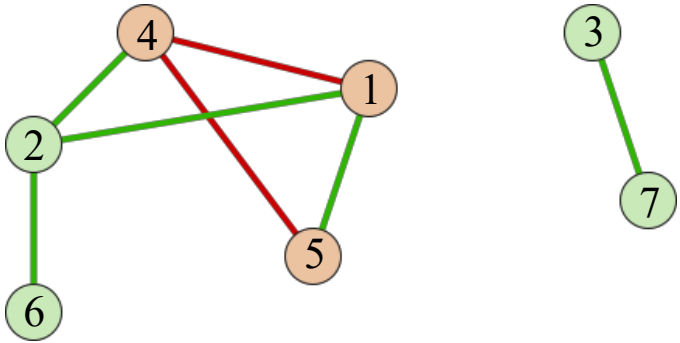
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(4, 5)
(4, 2)
(1, 2)
(1, 5)

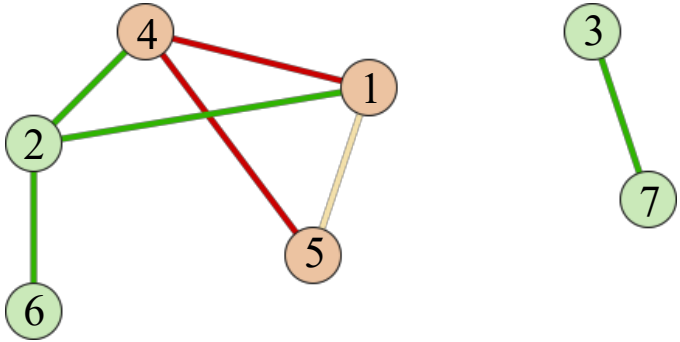
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(5, 1)
(5, 4)
(4, 2)
(1, 2)
(1, 5)

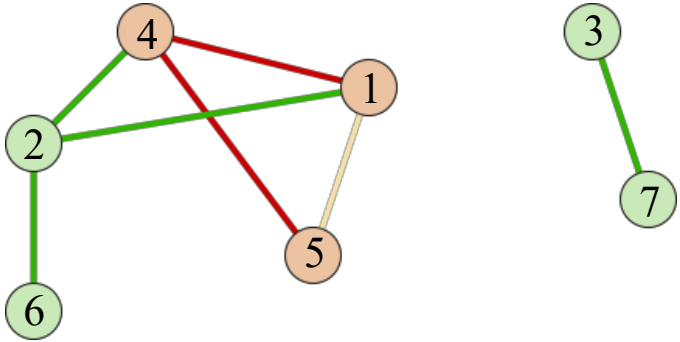
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(5, 4)
(4, 2)
(1, 2)
(1, 5)

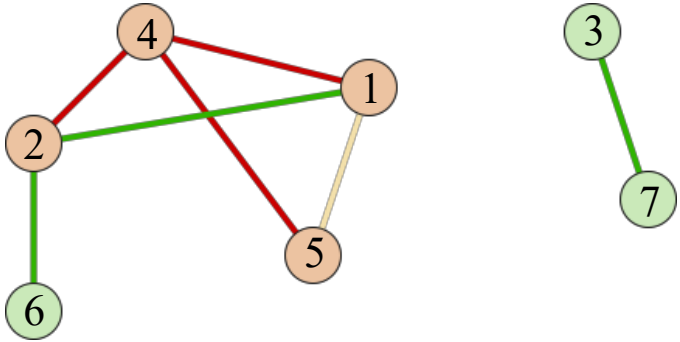
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(4, 2)
(1, 2)
(1, 5)

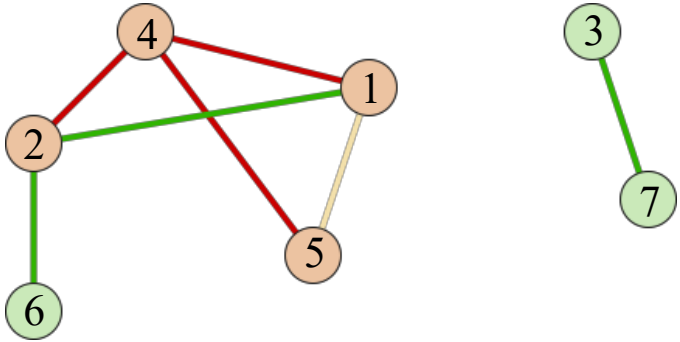
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(2, 4)
(2, 1)
(2, 6)
(1, 2)
(1, 5)

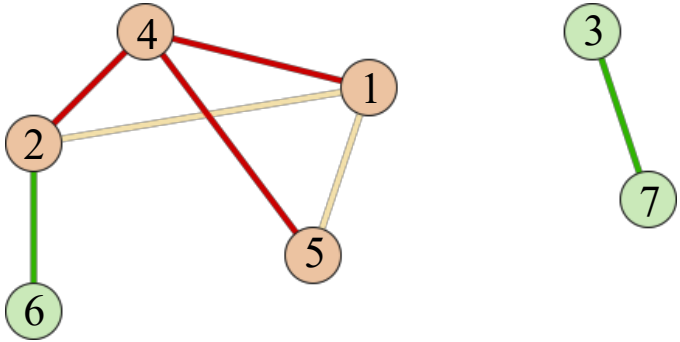
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(2, 1)
(2, 6)
(1, 2)
(1, 5)

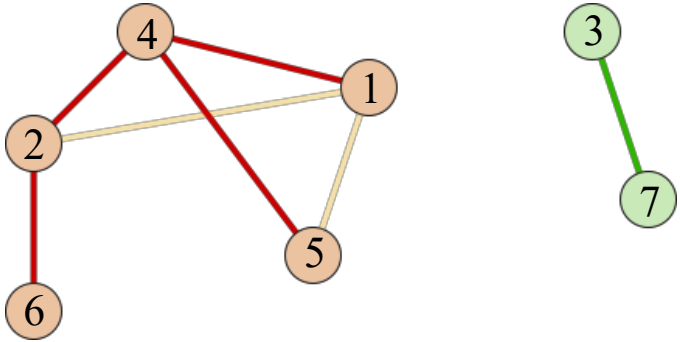
An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(2, 6)
(1, 2)
(1, 5)

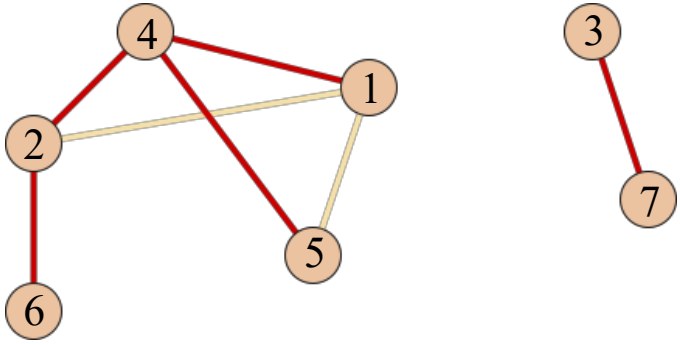
An Example



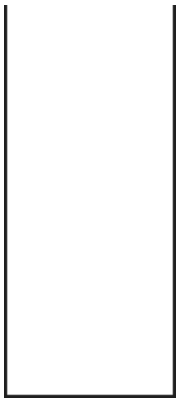

```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```

(6, 2)
(1, 2)
(1, 5)

An Example




```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7               then mark v as visited
8               S ← Adj(v)
9               while S is not empty
10                      do (u, w) ← Pop(S)
11                      if (u, w) is unexplored
12                              then if w is unexplored
13                              then mark edge (u, w) as tree edge
14                              mark vertex w as visited
15                              Push(S, Adj(w))
16                      else mark edge (u, w) as a non-tree edge
```



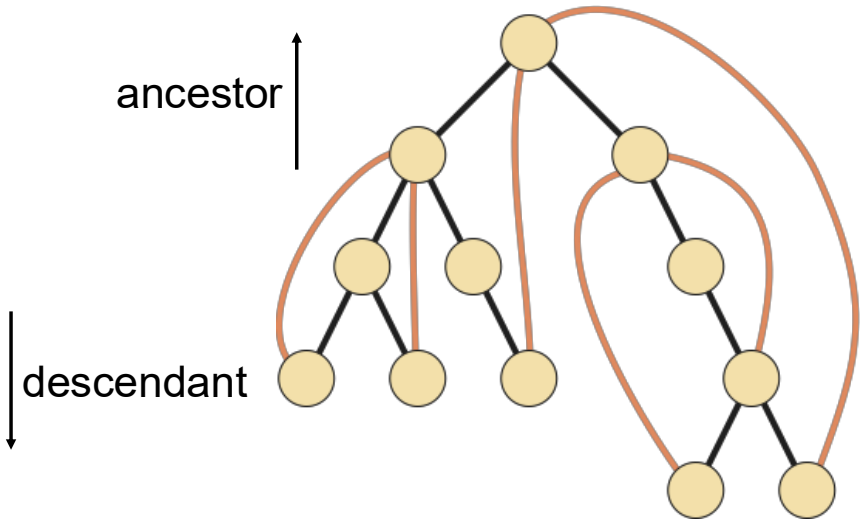
A Recursive DFS Algorithm

```
DFS(G)
1       for every vertex v of G
2               do label v as unexplored
3       for every edge e of G
4               do label e as unexplored
5       for every vertex v of G
6               do if v is unexplored
7                       then DFS(G, v)

DFS(G, v)
1       mark vertex v as visited
2       while every edge (v, w) in Adj(v)
3               do if (v, w) is unexplored
4               then if w is unexplored
5                       then mark edge (v, w) as tree edge
6                       DFS(G, w)
7       else mark edge (v, w) as a non-tree edge
```

An Important Property of Depth-First Search

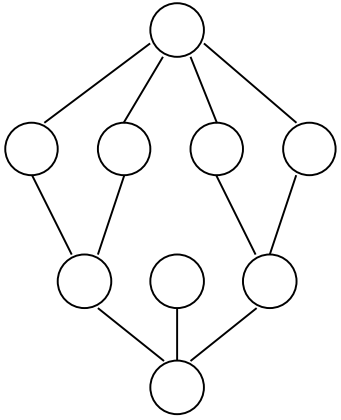
Theorem: *For every non-tree edge (v, w) in a depth-first spanning tree of an undirected graph, v is an ancestor of w or vice versa.*



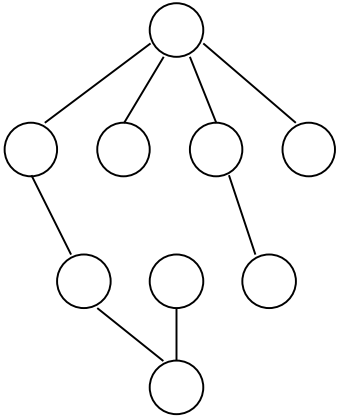
Graphs: Minimum Spanning Tree

Finding a spanning tree

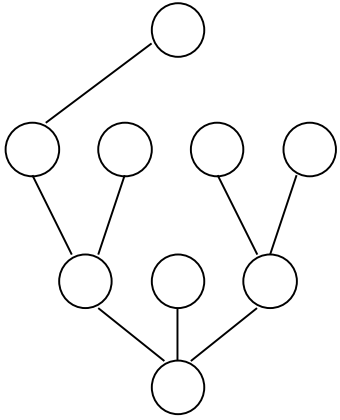
- To find a spanning tree of a graph,
  - pick an initial node and call it part of the spanning tree
  - do a search from the initial node:
    - each time you find a node that is not in the spanning tree, add to the spanning tree both the new node *and* the edge you followed to get to it



An undirected graph



One possible result of a BFS starting from top

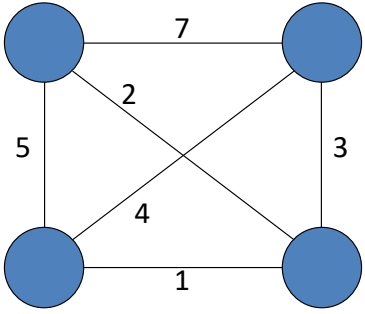


One possible result of a DFS starting from top

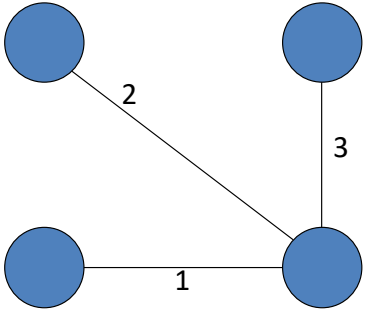
Minimum Spanning Trees

The Minimum Spanning Tree for a given graph is the Spanning Tree of minimum cost for that graph.

Complete Graph

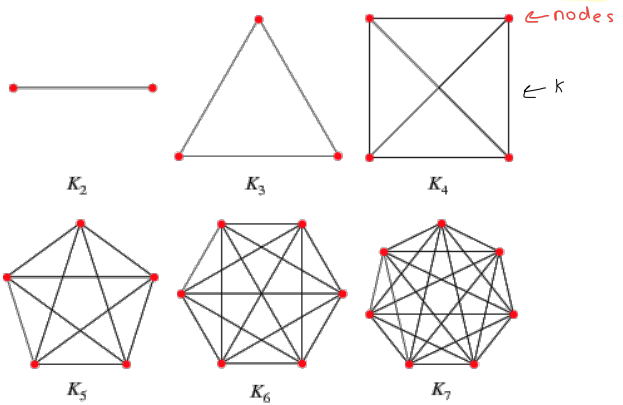


Minimum Spanning Tree



Complete Graph

- A complete graph is a graph in which each pair of graph vertices is connected by an edge. (every node is connected to every other node)
- The complete graph with **n** graph vertices is denoted $\mathbf{K_n}$ and has: $\binom{n}{2} = n(n-1)/2$ undirected edges, where $\binom{n}{2}$ is a binomial coefficient.
- The binomial coefficient $\binom{n}{k}$ is the number of ways of picking **k** unordered outcomes from **n** possibilities, also known as a combination or combinatorial number.
- The value of the binomial coefficient for nonnegative **n** and **k** is given explicitly by: $\binom{n}{k} = \frac{n!}{(n-k)!k!}$

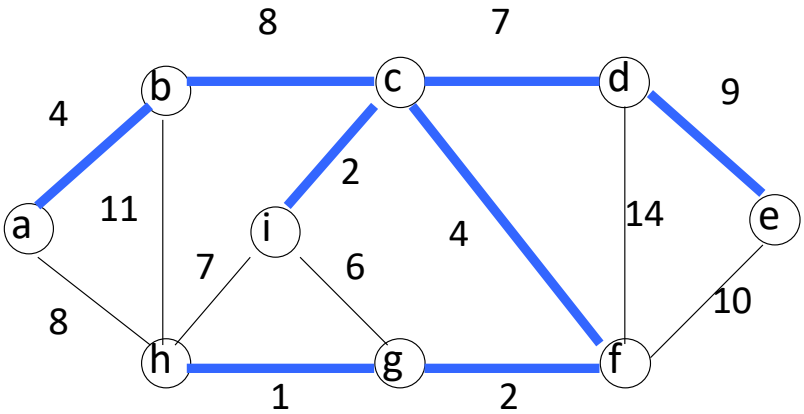


Minimum Spanning Tree (MST)

A **minimum spanning tree** is a subgraph of an undirected weighted graph ***G***, such that

- it is a tree (i.e., it is acyclic)
- it covers all the vertices ***V***
  - contains ***|V| - 1*** edges
- the total cost associated with tree edges is the minimum among all possible spanning trees
- not necessarily unique

Here is an example of a connected graph
and its minimum spanning tree:



Notice that the tree is not unique:
replacing (b,c) with (a,h) yields another spanning tree
with the same minimum weight.

Finding Minimum Spanning Trees

- There are two basic algorithms for finding minimum-cost spanning trees, and both are greedy algorithms

- **Kruskal's algorithm:** Start with *no* nodes or edges in the spanning tree, and repeatedly add the cheapest edge that does not create a cycle
  - Here, we consider the spanning tree to consist of edges only

- **Prim's algorithm:** Start with any *one node* in the spanning tree, and repeatedly add the cheapest edge, and the node it leads to, for which the node is not already in the spanning tree.
  - Here, we consider the spanning tree to consist of both nodes and edges

Kruskal's algorithm

```
T = empty spanning tree;
E = set of edges;
N = number of nodes in graph;
while T has fewer than N - 1 edges {
    remove an edge (v, w) of lowest cost from E
    if adding (v, w) to T would create a cycle
        then discard (v, w)
        else add (v, w) to T
}
```

- Finding an edge of lowest cost can be done just by sorting the edges
- Kruskal is O(E log E) as the edges have to be sorted first.

Prim's algorithm

```
T = a spanning tree containing a single node s;
E = set of edges adjacent to s;
while T does not contain all the nodes {
    remove an edge (v, w) of lowest cost from E
    if w is already in T then discard edge (v, w)
    else {
        add edge (v, w) and node w to T
        add to E the edges adjacent to w
    }
}
```

Application of MST: an example

- In the design of electronic circuitry, it is often necessary to make a set of pins electrically equivalent by wiring them together.
- To interconnect n pins, we can use n-1 wires, each connecting two pins.
- We want to minimize the total length of the wires.
- Minimum Spanning Trees can be used to model this problem.